Numeric Units

Integers

Unit of Data	Bit Width	Unsigned Range	Signed Range
Byte	8-bit / 2^8	0 to 255	-128 to $+127$
Half-word	16-bit / 2^{16}	0 to 65,535	$-32,768$ to $+32,767$
Word	32-bit / 2^{32}	0 to 4,294,967,295	$-2,147,483,648$ to $+2,147,483,647$

Floating-Point

Unit of Data	Bit Width	Range
Float	32-bit / 2^{32}	-3.403×10^{38} to 3.403×10^{38}
Double	64-bit / 2^{64}	-1.798×10^{308} to 1.798×10^{308}

Data Types

Syntax	Abbrev.	Description
`.byte <val1>, ...`	b	8-bit integer(s)
`.half <val1>, ...`	h	16-bit integer(s)
`.word <val1>, ...`	w	32-bit integer(s)
`.float <val1>, ...`	s	32-bit single-precision float(s)
`.double <val1>, ...`	d	64-bit double-precision float(s)
`.ascii "<string>"`	a	Non-null-terminated string
`.asciiz "<string>"`	a	Null-terminated string
`.space <bytes>`	a	Uninitialized byte allocation

Registers

Internal Register File

Symbolic Name	Register	Description
`$zero`	`$0`	Constant 0
`$at`	`$1`	Assembler temporary
`$v0 – $v1`	`$2 – $3`	Syscall codes & function returns
`$a0 – $a3`	`$4 – $7`	Function arguments
`$t0 – $t7`	`$8 – $15`	Temporary (not preserved)
`$s0 – $s7`	`$16 – $23`	Saved (preserved across calls)
`$t8 – $t9`	`$24 – $25`	Additional temporary (not preserved)
`$k0 – $k1`	`$26 – $27`	OS kernel reserved
`$gp`	`$28`	Global pointer (static data)
`$sp`	`$29`	Stack pointer
`$fp` (QTSpim: `$s8`)	`$30`	Frame pointer
`$ra`	`$31`	Return address
`$lo`	-	Mul/div lower 32 bits
`$hi`	-	Mul/div upper 32 bits

Floating-Point Unit (FPU / Coprocessor 1)

Register	Description
`$f0 – $f3`	Function returns
`$f4 – $f11`	Temporary (not preserved)
`$f12 – $f15`	Function arguments
`$f16 – $f19`	Additional temporary (not preserved)
`$f20 – $f31`	Saved (preserved across calls)

System Control Coprocessor (Coprocessor 0)

Symbolic Name	Register	Description
`$status` (or `$psw`)	`$12`	Processor status
`$cause`	`$13`	Exception cause
`$epc`	`$14`	Exception program counter
`$pc`	-	Program counter (next instruction)

Syscalls

Print Services

`$v0` Code	Type	Arguments
1	Integer	`$a0` = Integer
2	Float	`$f12` = Float
3	Double	`$f12` = Double
4	String	`$a0` = String address
11	Character	`$a0` = Character
34	Integer (Hex)	`$a0` = Integer
35	Integer (Binary)	`$a0` = Integer
36	Integer (Unsigned)	`$a0` = Integer

Read Services

`$v0` Code	Type	Arguments	Returns
5	Integer	–	`$v0` = Integer
6	Float	–	`$f0` = Float
7	Double	–	`$f0` = Double
8	String	`$a0` = Buffer `$a1` = Max bytes	–
12	Character	–	`$v0` = Character

File I/O Services

`$v0` Code	Name	Arguments	Returns
13	Open	`$a0` = Path `$a1` = Flags `$a2` = Mode	`$v0` = File descriptor (`<0` err)
14	Read	`$a0` = FD `$a1` = Buffer `$a2` = Max bytes	`$v0` = Bytes read (`0` EOF, `<0` err)
15	Write	`$a0` = FD `$a1` = Buffer `$a2` = Max bytes	`$v0` = Bytes written (`0` EOF, `<0` err)
16	Close	`$a0` = File descriptor	–

Program Services

`$v0` Code	Name	Arguments	Returns
9	Heap Alloc (sbrk)	`$a0` = Bytes to allocate	`$v0` = Allocated address
10	Exit	–	–
17	Exit (w/ Status)	`$a0` = Status code	–

Timing Services

`$v0` Code	Name	Arguments	Returns
30	System Time	–	`$a0` = Low 32 bits `$a1` = High 32 bits
32	Sleep	`$a0` = Milliseconds	–

Random Number Generation (RNG) Services

`$v0` Code	Type	Arguments	Returns
40	Set Seed	`$a0` = Generator ID `$a1` = Seed	–
41	Random Integer	`$a0` = Generator ID	`$a0` = Random Integer (Signed 32-bit range)
42	Random Integer Range	`$a0` = Generator ID `$a1` = Upper Bound	`$a0` = Random Integer (`0 <= x < $a1`)
43	Random Float	`$a0` = Generator ID	`$f0` = Random Float (`0.0 <= x < 1.0`)
44	Random Double	`$a0` = Generator ID	`$f0` = Random Double (`0.0 <= x < 1.0`)

Operands

Integer Operands

Operand Notation	Description
`Rdest`	Destination integer register
`Rsrc`	Source integer register
`Src`	Source integer register or immediate

Floating-Point Operands

Operand Notation	Description
`FRdest`	Destination floating-point register
`FRsrc`	Source floating-point register

Miscellaneous Operands

Operand Notation	Description
`Imm`	Immediate (decimal or hex)
`Mem`	Memory (label or address)

Internal Register File Instructions

System Operations

Instruction	Description
`nop`	No operation. Does nothing (advances program counter).
`syscall`	System call. Invokes OS service specified by `$v0`.

Load and Store

Instruction	Description
`l<type> Rdest, Mem`	Load from `Mem` to `Rdest`
`la Rdest, Mem`	Load address: `Rdest = &Mem`
`li Rdest, Imm`	Load immediate: `Rdest = Imm`
`lui Rdest, Imm`	Load upper immediate: `Rdest = Imm << 16`
`s<type> Rsrc, Mem`	Store `Rsrc` to `Mem`

Register Movement

Instruction	Description
`move Rdest, Rsrc`	Copy: `Rdest = Rsrc`
`mfhi Rdest`	Move from `$hi`: `Rdest = $hi`
`mthi Rsrc`	Move to `$hi`: `$hi = Rsrc`
`mflo Rdest`	Move from `$lo`: `Rdest = $lo`
`mtlo Rsrc`	Move to `$lo`: `$lo = Rsrc`

Conditional Register Movement

Instruction	Description
`movz Rdest, Rsrc1, Rsrc2`	Move if zero. `Rdest = Rsrc1` if `Rsrc2 == 0`.
`movn Rdest, Rsrc1, Rsrc2`	Move if not zero. `Rdest = Rsrc1` if `Rsrc2 != 0`.

Addition

Instruction	Description
`add Rdest, Rsrc, Src`	Signed: `Rdest = Rsrc + Src`
`addi Rdest, Rsrc, Imm`	Signed immediate: `Rdest = Rsrc + Imm`
`addu Rdest, Rsrc, Src`	Unsigned: `Rdest = Rsrc + Src`
`addiu Rdest, Rsrc, Imm`	Unsigned immediate: `Rdest = Rsrc + Imm`

Subtraction

Instruction	Description
`sub Rdest, Rsrc, Src`	Signed: `Rdest = Rsrc - Src`
`subu Rdest, Rsrc, Src`	Unsigned: `Rdest = Rsrc - Src`

Multiplication

Instruction	Description
`mult Rsrc1, Rsrc2`	Signed 64-bit: `$hi` / `$lo` = `Rsrc1 * Rsrc2`
`multu Rsrc1, Rsrc2`	Unsigned 64-bit: `$hi` / `$lo` = `Rsrc1 * Rsrc2`
`mul Rdest, Rsrc, Src`	Signed 32-bit: `Rdest = Rsrc * Src`
`mulo Rdest, Rsrc, Src`	Signed 32-bit (w/ overflow): `Rdest = Rsrc * Src`
`mulu Rdest, Rsrc, Src`	Unsigned 32-bit: `Rdest = Rsrc * Src`
`mulou Rdest, Rsrc, Src`	Unsigned 32-bit (w/ overflow): `Rdest = Rsrc * Src`

Division

Instruction	Description
`div Rsrc1, Rsrc2`	Signed: `$lo = Rsrc1 / Rsrc2`, `$hi = Rsrc1 % Rsrc2`
`divu Rsrc1, Rsrc2`	Unsigned: `$lo = Rsrc1 / Rsrc2`, `$hi = Rsrc1 % Rsrc2`
`div Rdest, Rsrc, Src`	Signed: `Rdest = Rsrc / Src`
`divu Rdest, Rsrc, Src`	Unsigned: `Rdest = Rsrc / Src`

Miscellaneous Arithmetic

Instruction	Description
`abs Rdest, Rsrc`	Absolute: `Rdest = |Rsrc|`
`neg Rdest, Rsrc`	Signed negate: `Rdest = -Rsrc`
`rem Rdest, Rsrc, Src`	Signed remainder: `Rdest = Rsrc % Src`
`remu Rdest, Rsrc, Src`	Unsigned remainder: `Rdest = Rsrc % Src`

Bitwise AND

Instruction	Description
`and Rdest, Rsrc, Src`	`Rdest = Rsrc & Src`
`andi Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc & Imm`

Bitwise OR

Instruction	Description
`or Rdest, Rsrc, Src`	`Rdest = Rsrc | Src`
`ori Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc | Imm`

Bitwise NOR

Instruction	Description
`nor Rdest, Rsrc, Src`	`Rdest = ~(Rsrc | Src)`

Bitwise XOR

Instruction	Description
`xor Rdest, Rsrc, Src`	`Rdest = Rsrc ^ Src`
`xori Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc ^ Imm`

Miscellaneous Bitwise

Instruction	Description
`clo Rdest, Rsrc`	Count leading ones. `Rdest` = number of consecutive 1s starting from MSB.
`clz Rdest, Rsrc`	Count leading zeros. `Rdest` = number of consecutive 0s starting from MSB.
`not Rdest, Rsrc`	Logical NOT: `Rdest = ~Rsrc`

Left Logical Bit Shift

Instruction	Description
`sll Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc << Imm` (zero-fill)
`sllv Rdest, Rsrc, Src`	Variable: `Rdest = Rsrc << Src` (zero-fill)

Right Logical Bit Shift

Instruction	Description
`srl Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc >> Imm` (zero-fill)
`srlv Rdest, Rsrc, Src`	Variable: `Rdest = Rsrc >> Src` (zero-fill)

Shift Right Arithmetic

Instruction	Description
`sra Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc >> Imm` (sign-extend)
`srav Rdest, Rsrc, Src`	Variable: `Rdest = Rsrc >> Src` (sign-extend)

Shift Rotation

Instruction	Description
`rol Rdest, Rsrc, Src`	Variable: `Rdest = Rsrc` rotated left by `Src` bits
`rol Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc` rotated left by `Imm` bits
`ror Rdest, Rsrc, Src`	Variable: `Rdest = Rsrc` rotated right by `Src` bits
`ror Rdest, Rsrc, Imm`	Immediate: `Rdest = Rsrc` rotated right by `Imm` bits

Unconditional Jumps

Instruction	Description
`j Mem`	Jump to `Mem`
`jr Rsrc`	Jump to address in `Rsrc`
`jal Mem`	Jump to `Mem`, save return in `$ra`
`jalr Rsrc`	Jump to address in `Rsrc`, save return in `$ra`

Unconditional Branching

Instruction	Description
`b Mem`	Branch unconditionally to `Mem`

Conditional Branching

Instruction	Description
`beq Rsrc1, Rsrc2, Mem`	Branch if `Rsrc1 == Rsrc2`
`bne Rsrc1, Rsrc2, Mem`	Branch if `Rsrc1 != Rsrc2`
`blt Rsrc1, Rsrc2, Mem`	Signed: Branch if `Rsrc1 < Rsrc2`
`bltu Rsrc1, Rsrc2, Mem`	Unsigned: Branch if `Rsrc1 < Rsrc2`
`ble Rsrc1, Rsrc2, Mem`	Signed: Branch if `Rsrc1 <= Rsrc2`
`bleu Rsrc1, Rsrc2, Mem`	Unsigned: Branch if `Rsrc1 <= Rsrc2`
`bgt Rsrc1, Rsrc2, Mem`	Signed: Branch if `Rsrc1 > Rsrc2`
`bgtu Rsrc1, Rsrc2, Mem`	Unsigned: Branch if `Rsrc1 > Rsrc2`
`bge Rsrc1, Rsrc2, Mem`	Signed: Branch if `Rsrc1 >= Rsrc2`
`bgeu Rsrc1, Rsrc2, Mem`	Unsigned: Branch if `Rsrc1 >= Rsrc2`

Zero Branching

Instruction	Description
`beqz Rsrc, Mem`	Branch if `Rsrc == 0`
`bnez Rsrc, Mem`	Branch if `Rsrc != 0`
`bltz Rsrc, Mem`	Branch if `Rsrc < 0`
`blez Rsrc, Mem`	Branch if `Rsrc <= 0`
`bgtz Rsrc, Mem`	Branch if `Rsrc > 0`
`bgez Rsrc, Mem`	Branch if `Rsrc >= 0`

Conditional Set Instructions

Instruction	Description
`slt Rdest, Rsrc1, Rsrc2`	Signed: `Rdest = 1` if `Rsrc1 < Rsrc2`, else `0`
`sltu Rdest, Rsrc1, Rsrc2`	Unsigned: `Rdest = 1` if `Rsrc1 < Rsrc2`, else `0`
`slti Rdest, Rsrc, Imm`	Signed immediate: `Rdest = 1` if `Rsrc < Imm`, else `0`
`sltiu Rdest, Rsrc, Imm`	Unsigned immediate: `Rdest = 1` if `Rsrc < Imm`, else `0`
`seq Rdest, Rsrc1, Rsrc2`	Equal: `Rdest = 1` if `Rsrc1 == Rsrc2`, else `0`
`sne Rdest, Rsrc1, Rsrc2`	Not equal: `Rdest = 1` if `Rsrc1 != Rsrc2`, else `0`

Floating-Point Unit (FPU) Instructions

Load and Store

Instruction	Description
`l.<type> FRdest, Mem`	Load from `Mem` to `FRdest`
`s.<type> FRsrc, Mem`	Store `FRsrc` to `Mem`

Register Movement

Instruction	Description
`mov.<type> FRdest, FRsrc`	Copy: `FRdest = FRsrc`

Conditional Register Movement

Instruction	Description
`movt Rdest, Rsrc1, Imm`	Move if FP true. `Rdest = Rsrc1` if FP condition flag `Imm` is `1`.
`movf Rdest, Rsrc1, Imm`	Move if FP false. `Rdest = Rsrc1` if FP condition flag `Imm` is `0`.

Cross-Processor Register Movement

Instruction	Description
`mfc1 Rdest, FRsrc`	Move from FPU: `Rdest = FRsrc` (32-bit)
`mfc1.d Rdest, FRsrc`	Move double from FPU: `Rdest` & `Rdest+1 = FRsrc` (64-bit)
`mtc1 Rsrc, FRdest`	Move to FPU: `FRdest = Rsrc` (32-bit)
`mtc1.d Rsrc, FRdest`	Move double to FPU: `FRdest = Rsrc` & `Rsrc+1` (64-bit)

Data Representation Conversion

Instruction	Description
`cvt.<to>.<from> FRdest, FRsrc`	Format conversion. Converts `FRsrc` from `<from>` to `<to>`.
`ceil.w.<type> FRdest, FRsrc`	Ceil to word. Converts to 32-bit int (rounds up).
`floor.w.<type> FRdest, FRsrc`	Floor to word. Converts to 32-bit int (rounds down).
`round.w.<type> FRdest, FRsrc`	Round to word. Converts to 32-bit int (rounds to nearest).
`trunc.w.<type> FRdest, FRsrc`	Truncate to word. Converts to 32-bit int (rounds to zero).

Addition

Instruction	Description
`add.<type> FRdest, FRsrc1, FRsrc2`	`FRdest = FRsrc1 + FRsrc2`

Subtraction

Instruction	Description
`sub.<type> FRdest, FRsrc1, FRsrc2`	`FRdest = FRsrc1 - FRsrc2`

Multiplication

Instruction	Description
`mul.<type> FRdest, FRsrc1, FRsrc2`	`FRdest = FRsrc1 * FRsrc2`

Division

Instruction	Description
`div.<type> FRdest, FRsrc1, FRsrc2`	`FRdest = FRsrc1 / FRsrc2`

Miscellaneous Arithmetic

Instruction	Description
`abs.<type> FRdest, FRsrc`	Absolute: `FRdest = |FRsrc|`
`neg.<type> FRdest, FRsrc`	Negate: `FRdest = -FRsrc`
`sqrt.<type> FRdest, FRsrc`	Square root: `FRdest = sqrt(FRsrc)`

Conditional Branching

Instruction	Description
`c.<cond>.<type> FRsrc1, FRsrc2`	Compare. Sets FP flag if `FRsrc1 <cond> FRsrc2`. (`<cond>` options: `eq`, `lt`, `le`)
`bc1t Mem`	Branch if FP true. Jumps to `Mem` if the FP flag is `1`.
`bc1f Mem`	Branch if FP false. Jumps to `Mem` if the FP flag is `0`.